Care Note — Technical Brief

Nightingale 72-Hour Build. One shared, longitudinal, role-based patient record combining clinician notes, staff notes, patient insight and AI-scribed consult summaries — with a Glance View, full provenance, revision history and server-enforced RBAC. **Synthetic data only; not safe for real PHI as-is** (§6).

1. Thesis

The brief's hardest sentence is a question: *we trust LLMs up to a point, then we need reassurance from clinicians.* A summariser is easy; one a clinician should rely on mid-consult is the actual problem. Everything follows from one position — **AI output enters this system as a claim, not a fact.** A claim has a source you can open, a confidence you can see, a human who can accept or reject it, and no power to overwrite what a clinician wrote. That is a data-model commitment before a UI one, which is why it is in the Phase 0 schema.

2. Architecture

```
Browser (React SPA) ── UI gating is convenience only; assumed compromised
      │  httpOnly cookie (SameSite=lax, 60min) — no token in JS
      ▼
FastAPI
 ├─ require_access(*roles)  ◄── THE boundary. Verifies JWT, checks role,
 │    yields AccessScope(user, role, clinic_id, db) — and nothing else.
 │    No bare User, no bare Session escapes to a handler.
 ├─ Route handlers
 │    scope.query(Model)           → clinic filter applied HERE
 │    scope.assert_can_write_type() → policy.py matrix
 ├─ llm_client.complete()   ◄── ONLY module that reaches a model
 │    redact_phi() → find_residual_phi() → raise PHILeakError, or send
 └─ asr_client.transcribe()  ◄── ONLY audio→text path; fail-closed egress gate
      ▼                     ▼
 SQLite / Postgres    LLM provider (offline stub by default)

Audit + interaction logging alongside: IDs, actions, timestamps only.
```

Python 3.11 / FastAPI / SQLAlchemy / SQLite (Postgres-ready), React 18 + Vite + Tailwind, pytest. FastAPI specifically because dependency injection is what makes the RBAC rule unforgettable:

RBAC fuses role and clinic inseparably. The common real failure is a route that checks role and forgets the tenant filter. Here that is not expressible. `require_access()` is the only way a route learns its caller — there is no exported `get_current_user`. It yields an `AccessScope`, never a `User`, and that is also the only DB handle a route gets. `scope.query()` applies the clinic predicate itself and **raises `TypeError`** on any model lacking `clinic_id` rather than returning it unfiltered. `clinic_id` comes from the verified JWT only. No handler in `patient_routes.py` mentions `clinic_id` in a filter. Cross-clinic fetches by exact id return **404, not 403** — a 403 confirms the id exists.

One redaction chokepoint. `redact_phi()` is called unconditionally by `complete()`, the only module that reaches a model. Callers cannot opt out; redaction is idempotent. After redacting, the payload is re-scanned and the call **raises rather than sends** if PHI survived. Three source-scanning tests fail the build if any other module imports an LLM SDK, references a model endpoint, or removes the redaction call.

3. Schema

`Entry` is the hub. Every relationship the brief names hangs off it.

Link	Mechanism
Entry → Versions	`Version.entry_id`, unique `(entry_id, version_number)`. Full snapshots, not diffs — revert is a copy, and no chain can be corrupted by one bad link
Entry → Comments	`Comment.entry_id`; threads via self-referential `parent_comment_id`; open/resolved status
Entry → Highlights	`entry_id` + character span + **the `source_version_number` the span was computed against**, so an edit cannot silently move a highlight onto different text
Entry → AIScribedNote	One-to-one. This row's *presence* is what makes an entry AI-authored; `author_role='system'` is the denormalised fast check. A UI cannot render an AI note as a clinician's
AIScribedNote → transcript	Shared `session_id` with `TranscriptSegment`
Summary line → spoken words	`SummaryAttribution` links each summary line to the segment that produced it
Anything → Provenance	A **string URI**, not a foreign key

`CLINIC ─< USER, PATIENT, ENTRY, FEATURE_WEIGHT` · `PATIENT ─< ENTRY, TASK, CAPTURE_SESSION` · `ENTRY ─< VERSION, COMMENT, HIGHLIGHT, TASK, SUMMARY_ATTRIBUTION` and ─o `AI_SCRIBED_NOTE, ENTRY_ARCHIVE` · `COMMENT ─< COMMENT, TASK` · `USER ─< INTERACTION_LOG, AUDIT_LOG, PATIENT_VIEW`. Full Mermaid ER diagram in `SCHEMA.md`.

Provenance is a URI — entry://<id>#span:<start>-<end>, session://<id>#turn:<n>, transcript://<id>#segment:<n>. A foreign key points at one table; provenance targets are heterogeneous — a whole entry, a character range in one, a turn in an AI session, an audio segment that is not a row here. One resolvable string covers all and keeps "click a highlight, land on the source" to one code path. Cost: the DB won't enforce integrity, so `resolve()` is the only dereference path, it **raises on a dangling pointer** rather than degrading to empty, and it enforces `clinic_id` — a valid pointer must never become a cross-tenant read primitive.

`clinic_id` is denormalised onto every scoped table, even where a join would derive it. That redundancy is what lets `AccessScope.query()` apply one uniform predicate to any model — and why it can *refuse* one lacking the column.

Learning integration. `InteractionLog` records what clinicians touch as **extracted tags only, never prose**. `record_interaction()` calls `apply_signal()` in the same operation, so no route can log a signal the learning table never sees — the same chokepoint reasoning as redaction. Evidence aggregates per `(clinic_id, feature_tag)` into `FeatureWeight` with a 90-day half-life, saturating into (−1, 1), read by `scoring.learned_component()` as one term. `FeatureWeight` is a materialised view of the log, never nudged, so it cannot drift from its evidence. It is capped at 0.25 of the total, cannot invent a highlight (the rule layer runs first), cannot cross a clinic, cannot be trained by patients, and **cannot silence safety vocabulary** — allergy, sepsis, anaphylaxis and self-harm tags are floored at zero.

4. Glance View and latency

The Top Card answers four questions in fixed order: what changed since you were last here; what could hurt this patient; what matters and why; what is outstanding and whose it is. **Refusing to surface things is what makes it readable in ten seconds** — a Top Card showing everything is a timeline with extra steps. Ranking is a weighted sum over named features, each highlight showing its own arithmetic and a one-line `risk_reason`. Nothing is ranked by a model.

Measured: P95 ≤ 300 ms target. A middleware reports `X-Response-Time-Ms` per request — request in, queries run, payload serialised, response out. 200 iterations after 20 discarded warm-ups, 10-entry chart with 6 highlights, SQLite on local disk. Three consecutive runs: **P95 14.26 / 13.30 / 15.94 ms.** Range reported, not the best run.

This **excludes network transit and browser render** — those depend on deployment and device, and folding loopback in would invent precision. The client measures its own round trip and the header shows both, so the demo never conflates them.

The figure is evidence for something narrower than "fast": **application work is a small fraction of budget** and no N+1 hides in the hot path. Two decisions carry that — **highlight scores are computed on write, not read**, so the view reads precomputed rows; and **timeline enrichment is batched** into four grouped queries regardless of chart size. Production means Postgres over a network, hundreds of entries and concurrent load; the ~20× headroom makes inversion unlikely, but the test that settles it is a loaded staging environment.

5. Trust calibration — three mechanisms

1. Accept / reject. `Highlight.status` starts `suggested` and needs a clinician decision; no AI claim reaches the card as fact on its own authority. One click, inline, no navigation, immediate confirmation — because this decision is also the training signal, and a high-friction control starves the loop. The interaction cost is a design constraint, not a nicety.

2. Visible confidence, derived not asserted. On the offline path confidence comes from hedging density in the source transcript: the patient session full of "maybe" lands near 0.47 and is flagged; the measurement-heavy nurse consult near 0.77. Low-confidence summaries flag **separately from risk** — "this might be dangerous" and "this might be wrong" are different warnings a clinician acts on differently. Uniform confidence in an interface is itself a claim, usually false.

3. Clinician precedence *and* a review flag. The brief allows either; we do both. A clinician edit wins immediately — care is never blocked on a resolution workflow — but `conflict_flagged` is set and `supersedes_entry_id` records what was overridden, so the disagreement stays visible. Precedence alone loses information: that the AI disagreed is *itself* clinically interesting, and discarding it quietly is how a system teaches users to stop trusting it. AI notes are never edited in place; corrections supersede.

Supporting all three: `provenance_pointer` is non-nullable on `Highlight`, resolution lands on the **character span** not the note, and AI-vs-human is carried by four independent signals (rail style, colour, typeface, label) so it survives in greyscale.

6. Trade-offs, assumptions, deferred scope

Regex redaction over NER (D-012). Data is synthetic, so recall against real name diversity isn't what's tested — the boundary's un-bypassability is. Regex is auditable line by line, worth more in a trust system than F1 from an uninspectable model. Production layers NER *behind the same signature*.

Content is deliberately not HTML-escaped on write (D-015). Clinical prose contains `BP <120/80` and `dose <5mg`. Escaping on write double-escapes on render; tag-stripping can eat `<5mg` and silently turn a dose limit into `mg`. Corrupting a note is worse than the XSS it prevents — because untrusted content is never rendered as HTML at all, enforced by a build-failing source scan.

Assumptions where the brief is silent: staff cannot view `clinician_sections` (D-004, least privilege); admin reads all in-clinic but authors no clinical content (D-011), so it cannot quietly alter the record.

Optimistic locking over CRDTs. RBAC already partitions who writes what, so most conflicts are prevented by construction; presence is polish paid for in infrastructure.

Multilingual: partially closed, and the design point matters (D-058). Phase 5 carried code-switched speech through intact but tagged English only, so identical clinical content produced `['symptom:swelling']` in English and `[]` in Malay — no tags, no score, never on the Glance View. That fails in the worst direction: the patients least likely to be understood in English are the ones the system stops surfacing. A Malay vocabulary now maps each term to the **canonical English tag**, so `bengkak` emits `symptom:swelling`. That is not cosmetic — tags are the keys Phase 4 learns against, and a separate `symptom:bengkak` would have split one concept into two features and stopped a clinic's learned attention transferring across the language its patients used. Scope is deliberately narrow: only terms whose English counterpart already exists, so no English input changes behaviour. Still unbuilt: translation, non-English summary generation, and every language but Malay. The fourteen terms need native-speaker and clinical review — the mechanism is proven, the word list is a demonstration. Testing it also surfaced that **negation is unhandled in both languages** ("Patient denies chest pain" tags chest pain); pre-existing, now pinned by tests in both languages so a fix cannot be applied to one and not the other.

Deferred: multilingual summary generation and handwriting capture (D-019). Summary generation in a second language is a *time* deferral only — the path is a second-language summary from the existing LLM call. Handwriting OCR was deferred **structurally**: a different ingestion pipeline (image → OCR → redact → summarise) where redaction is materially harder on noisy output — one misrecognised character defeats a pattern that would have caught an identifier — and medical handwriting OCR is hard even for well-resourced products. Ambient voice capture serves the same "fast unstructured capture" need more safely.

One defect worth reporting. The final pass found every enum column is declared `Mapped[StrEnum]` but backed by `String`, so reloaded rows return plain `str` and three `is` comparisons were silently dead branches (D-055) — most visibly, superseded highlight suggestions were never deleted and the Top Card rendered every claim twice. It was live through 334 passing tests and was found by *looking at the screen*. Fixed and pinned by regressions plus a source scan; the column-type migration is deferred rather than attempted hours before submission. Reported because the honest version of "structural enforcement catches mistakes" includes the class it missed.

7. Security posture

Implemented = built, tested, verifiable here. **Documented decision** = deliberate for a 72-hour prototype, production shape stated. **Known gap** = genuinely missing, listed because a control whose edges nobody knows is worse than a weaker one everybody understands.

Area	Status
PHI redaction chokepoint	**Implemented** — regex + gazetteer, fail-closed
Stored-XSS / content safety	**Implemented** — never rendered as HTML, enforced by source scan
RBAC (role + clinic, server-side)	**Implemented** — fused, proven over HTTP
Logging hygiene	**Implemented** — content-free by construction, verified by grep
JWT storage	**Implemented** — httpOnly cookie, SameSite=lax, 60min TTL
AI note immutability; comment isolation from patients	**Implemented** — corrections supersede; comments refused at route *and* stamped internal
Conflict handling	**Implemented** — precedence *and* flag; disputed content never deleted
Audio never persisted; un-redacted egress	**Implemented** — memory only, asserted against the DB; egress gate fails closed, never degrades to stub
Learning substrate holds no prose; clinic-partitioned; safety floored	**Implemented**
Enum comparison correctness	**Implemented (guarded)** — **==** throughout; source scan fails the build (D-055)
CSRF defence	**Documented decision** — `SameSite=lax` only
TLS in transit	**Documented decision** — terminates at reverse proxy / LB with HSTS in production; **not implemented locally (plain HTTP)**
Encryption at rest	**Documented decision** — managed-Postgres volume encryption or SQLCipher in production; **SQLite here is unencrypted**
Password hashing; decay scheduling	**Documented decision** — PBKDF2 120k (argon2id in prod); explicit trigger, no cron
Token refresh / rotation / revocation	**Known gap** — no refresh flow, no denylist
Login rate limiting	**Known gap**
Redaction recall on unanticipated names	**Known gap** — lowercase/transliterated names in prose can survive
Scribe failure recovery	**Known gap** — synchronous; a crash mid-run loses the summary
Real speech recognition	**Known gap** — default recogniser is a **simulated stub**; no audio has ever been transcribed by this build
Acoustic diarisation; consent artefact on patient recordings	**Known gap** — labels come from the transcript source; the clinician is a party and is never asked
Per-user normalisation of learning signals	**Known gap** — one enthusiast counts as consensus
Enum columns typed `String` not `Enum`	**Known gap** — structural fix for D-055
Formal accessibility / WCAG audit	**Known gap** — colour is never the sole signal, but no audit was run

Plainly: locally there is no TLS and no encryption at rest — plain HTTP on localhost, an unencrypted gitignored SQLite file. Both are deployment configuration rather than application code, hence decisions with a stated production shape; the honest consequence is that **this build is not safe for real PHI as-is**, which the README states too so it cannot be missed by someone who opens one file.

Verification. 385 tests, no API key or network needed. Access-control and history tests were **deliberately broken to confirm they can fail** — reversing D-004 fails exactly the staff-visibility tests, removing the clinic filter fails 15, disabling the conflict guard fails 3. A security test that cannot fail is worse than none, because it is mistaken for coverage. Logs were grepped for planted names, identifiers and body text after exercising every route: zero hits.